

TP 2 Java FX : Création d'une interface utilisateur en utilisant FXML

I- Objectif

Dans cet atelier, vous allez apprendre à utiliser FXML, un langage basé sur XML fourni par JavaFX, pour créer l'interface utilisateur de votre application Desktop

FXML vous permet d'écrire l'interface utilisateur séparément de la logique d'application, facilitant ainsi la maintenance du code.

Dans l'atelier précédent dans JavaFX, on a créé une interface graphique de formulaire d'authentification en utilisant du code Java.

Dans cet atelier, on va voir comment créer un formulaire d'inscription à l'aide de FXML.

La figure ci-dessous montre l'application de formulaire d'inscription que nous allons créer à l'aide de FXML

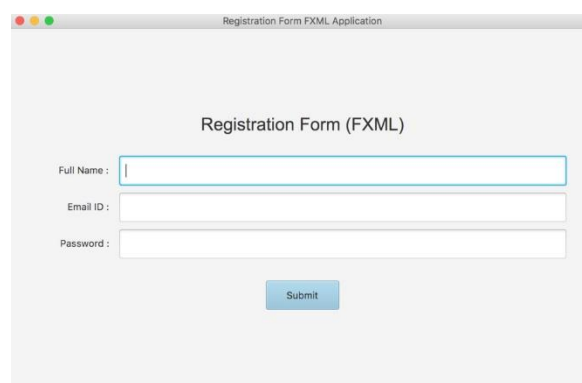


Fig1 formulaire d'inscription

II- Structure d'une application FXML

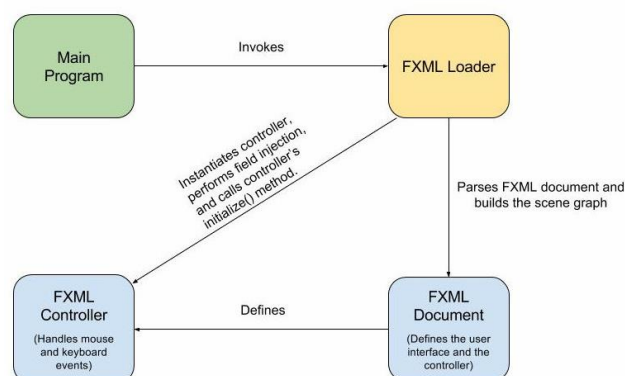


Fig2 : Structure d'une application FXML

Comme illustré sur la figure fig2, l'interface utilisateur d'une application FXML est définie dans un document FXML et toute la logique pour gérer les événements d'entrée est écrite dans une classe de contrôleur.

L'exécution du programme commence par la classe Main, qui invoque le loader FXML qui lui-même analyse le document FXML, instancie les nœuds spécifiés dans le document et crée la javafx **Scene**.

Après avoir créé le graphique de scène, le chargeur FXML instancie la classe de contrôleur, injecte les champs définis dans la classe de contrôleur avec des objets instanciés à partir du document FXML, puis appelle la méthode initialize () du contrôleur.

Pour cela dans une application on aura besoin au moins de 3 fichiers un fichier fxml dans lequel on va créer l'interface un autre fichier controller dans lequel on va créer une classe qui contient des méthodes pour la gestion d'évènement et un autre fichier qui contient une classe application (start + main)

III- Création du code de l'application (formulaire d'inscription)

Dans l'application vous allez créer trois fichiers : **RegistrationFormApplication.java**, **RegistrationFormController.java** et **Registration_form.xml** dans un package nommé **javafx.example** comme le montre la figure suivante **Fig 3**

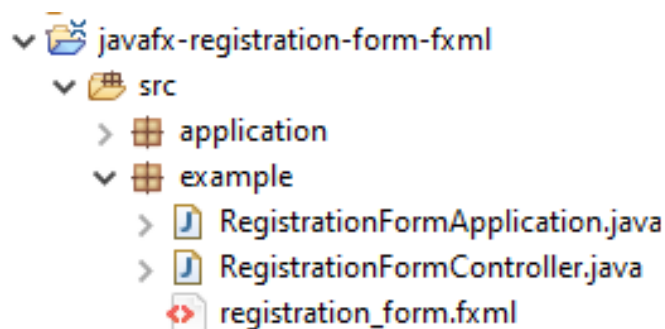


Fig3 : Arborescence de l'application

- `RegistrationFormApplication.java` contient la classe application Main.
- `registration_form.fxml` est le Document FXML qui contient l'interface utilisateur de l'application
- `RegistrationFormController.java` est le Controller FXML qui sera utilisé pour le code du gestionnaire d'événements.

IV- Création du document FXML en utilisant SceneBuilder

Pour se faire on va ouvrir le fichier document FXML en utilisant **SceneBuilder** comme le montre la figure suivante. Fig4

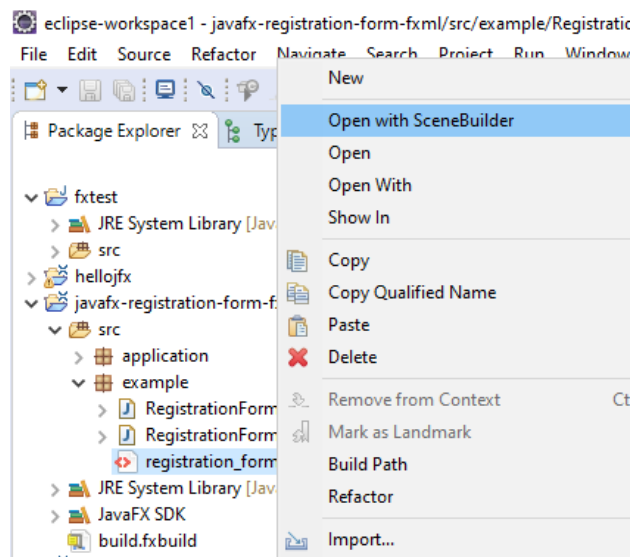


Fig 4 : création d'une application à l'aide du Scenbuilder

A partir de l'éditeur **Scenbuilder** vous pouvez créer votre interface en glissant les différents composants de l'interface qui se trouve à gauche et en précisant les propriétés de chaque composant

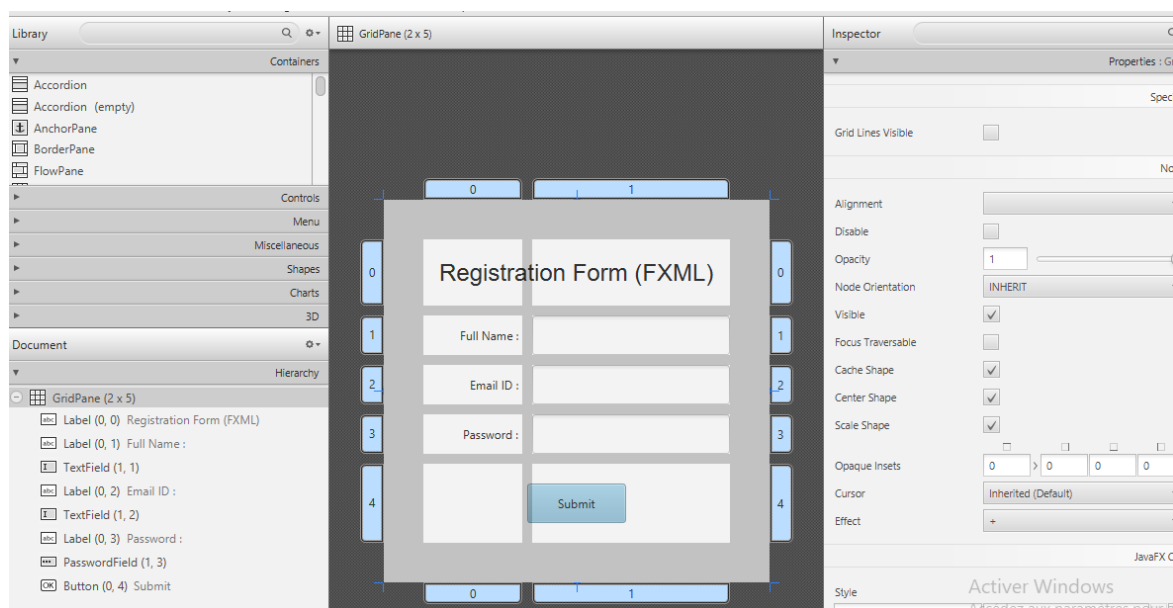


Fig5 : ajout des composants dans l'interface utilisateur

Après l'enregistrement de l'interface, du code FXML sera généré automatiquement dans le document `registration_form.fxml`

V- Chargement du document FXML dans la classe principale

Comme d'habitude, pour créer une application JavaFX, nous devons hériter la classe `javafx.application.Application` et implémenter sa méthode `start ()`.

```
package javafx.example;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class RegistrationFormApplication extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception{
        Parent root =
FXMLLoader.load(getClass().getResource("registration_form.fxml"));
        primaryStage.setTitle("Registration Form FXML Application");
        primaryStage.setScene(new Scene(root, 800, 500));
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Nous chargeons tous d'abord le document FXML à l'aide de **FXMLLoader**

=> **FXMLLoader.load(getClass().getResource("registration_form.fxml"));**

Cela nous donne une référence au nœud racine de l'interface utilisateur.

Après cela, on va créer une **Scene** en utilisant le nœud racine FXML et on va définir la scène dans la scène principale.

```
primaryStage.setScene(new Scene(root, 800, 500));
primaryStage.show();
```

VI- Création du code pour gérer l'événement Submit dans le contrôleur

Les trois champs doivent être bien remplis sinon une boîte de dialogue Alert s'affiche comme le montre les figures suivantes (Fig6 + Fig 7)



Fig6 : interface utilisateur inscription

Sinon



Fig7 : interface utilisateur inscription avec erreur de saisie

On va créer tout d'abord la classe **RegistrationFormController** qui contient une fonction pour la gestion de l'événement correspondant au bouton Submit

```
package javafx.example;

import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.PasswordField;
import javafx.scene.control.TextField;
import javafx.stage.Window;

public class RegistrationFormController {
    @FXML
    private TextField nameField;

    @FXML
    private TextField emailField;

    @FXML
    private PasswordField passwordField;

    @FXML
    private Button submitButton;
    @FXML
    protected void handleSubmitButtonAction(ActionEvent event)
    {
        Window owner = submitButton.getScene().getWindow();
        if(nameField.getText().isEmpty()) {

            Alert alert = new Alert(AlertType.ERROR);
            alert.setTitle("Form Error!");
            alert.setHeaderText(null);
            alert.setContentText("Please enter your name");
            alert.initOwner(owner);
            alert.show();

            //.....Vous faites le test pour tous les autres champs>
            //... .....

        }
        else

            "Welcome " + nameField.getText());
    }
}
```

Il faut s'assurer que les contrôles qui se trouvent dans le code source java portent le même nom que celui dans l'éditeur **Scene Builder**

VII- Travail à faire

- Créer les différentes classes de l'application
- Créer l'interface utilisateur à partir du SceneBuilder
- Compléter le code pour la gestion d'évènement dans le code la méthode **handleSubmitButtonAction** pour vérifier **si** les autres champs sont remplis ou non
- Faire le lien entre l'interface graphique et la classe contrôleur **RegistrationFormController** (voir annexe 1)

Annexe I

Pour faire le lien entre l'interface utilisateur à partir du Scenbuilder et le contrôleur il faut tout d'abord indiquer le nom de la classe `RegistrationFormController.java` dans la partie controller come le montre la figure Fig8

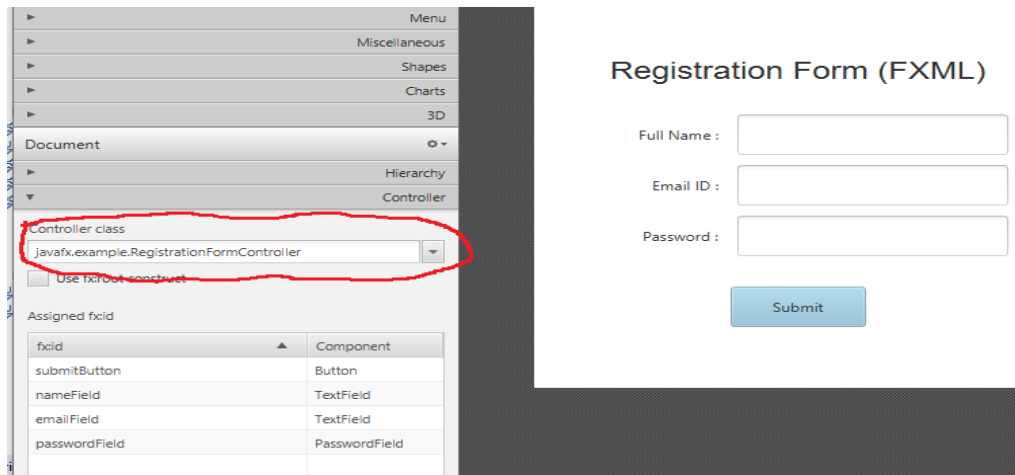


Fig8

Puis sélectionner le bouton Submit et préciser quelle est la méthode que va déclencher l'évènement **on Action** qui se trouve la rubrique **Code** et dans notre cas **handleSubmitButtonAction**

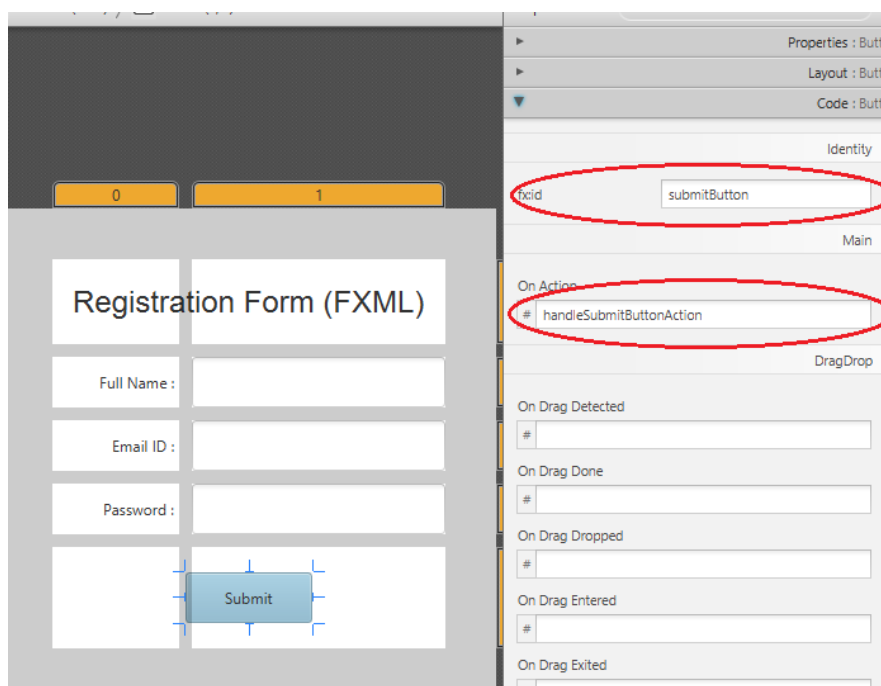


Fig 9